

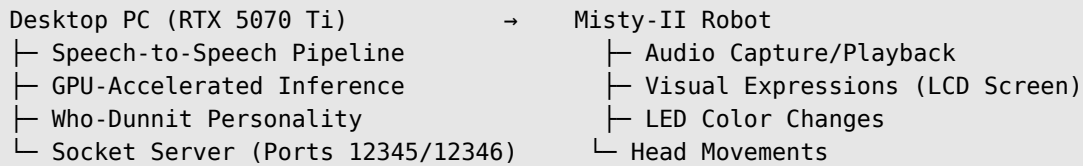
HRI Experiment Setup

Project Overview

This project implements a **social robotics experiment** where a Misty-II robot guides human participants through collaborative problem-solving tasks using real-time speech-to-speech interaction (+ more). The base system implements advanced human-robot interaction capabilities with latency monitoring and distributed AI processing.

Architecture

Distributed AI System



Pipeline Components

VAD → STT → LLM → TTS

1. **Voice Activity Detection:** Silero VAD v5 (real-time speech detection)
2. **Speech-to-Text:** Faster-Whisper (GPU-accelerated transcription)
3. **Language Model:** Microsoft Phi-3-mini-4k (conversational AI with who-dunnit personality)
4. **Text-to-Speech:** Parler-TTS (natural voice synthesis)

Experimental Design

Problem-Solving Tasks

The robot guides participants through **3 collaborative tasks** that revolve around solving a cryptic ‘who-dunnit’ mystery (where is the missing robot?). Tasks include:

1. **‘Suspect’ Identification:** Yes/no questions to identify a perpetrator from a 6×4 grid
2. **System Analysis:** Analyzing robot logs to determine functionality status
3. **Location Inference:** Using Wi-Fi logs + other logs to find location of the missing robot

Robot Personality

- **Adaptive behavior modes:** CONTROL (reactive) vs RESPONSIVE (proactive)
- **Affect detection:** Monitors user hesitation, confusion, frustration
- **Progressive hint system:** 3-tier scaffolding (H1→H2→H3)
- **Expression system:** 16 distinct robot expressions linked to conversation context (adding more once pipeline is stable)

JSON Response Architecture

```
{
  "msg": "I'm sensing hesitation—would you like a hint?",
```

```
"expression": "hint" # triggers robot to display 'thinking' expression (RGB
LED + head movement + LCD Screen + arm movement)
}
```

- **msg**: Speech content for TTS
- **expression**: Robot action trigger (LED, head movement, facial display)

Technical Implementation

Performance Optimizations

- **Python 3.12 environment** (resolves TensorFlow/protobuf compatibility)
- **CUDA acceleration** on RTX 5070 Ti
- **Distributed processing** (AI on PC, interaction on robot)
- **Streaming audio** for reduced latency

Latency Monitoring System

Component Breakdown:

- VAD: ~30-50ms (voice detection)
- STT: ~200-350ms (speech transcription)
- LLM: ~300-600ms (response generation for simple lookups)
- TTS: ~350-450ms (voice synthesis)
- Robot: ~200-300ms (expression changes + network)

Performance Targets:

- Total response time: <2000ms (natural conversation)
- Current performance: ~1150-1750ms (Grade A+ performance)

Network Architecture

- **Server mode**: Pipeline runs on desktop PC
- **Tailscale VPN**: Secure connection between laptop and desktop
- **Client connections**: Misty robot connects via WiFi to laptop
- **Audio streaming**: Real-time bidirectional audio over websockets

Features

1. Multimodal Robot Communication

- **Simultaneous speech and expression**: TTS plays while robot performs actions
- **Context-aware expressions**: Robot behavior matches conversation state
- **Affect-responsive interactions**: Proactive assistance based on user state

2. Structured Task Performance

- **Pre-defined knowledge base**: All answers available in system prompt
- **No complex reasoning required**: Optimized for fast, consistent responses
- **Progressive difficulty**: Guided scaffolding through hint system

3. Production-Ready Monitoring

- **Real-time latency tracking:** Component-level and end-to-end timing
- **Performance grading:** Automatic A-F rating system
- **Experiment data export:** JSON reports for analysis
- **Statistical analysis:** Mean, median, P95 response times

Development Environment

Dependencies

- **Core ML:** PyTorch, Transformers, TorchAudio
- **Speech Processing:** Faster-Whisper, Parler-TTS, Silero VAD
- **Robot Control:** Misty Python SDK
- **Networking:** Tailscale + Socket-based audio streaming
- **Monitoring:** Custom latency measurement system

File Structure

```
speech-to-speech/  
├─ s2s_pipeline.py           # Main pipeline orchestrator  
├─ who-dunnit-instruction.md # Complete robot roles, rules & tasks  
├─ misty_integration.py      # Robot hardware interface  
├─ latency_monitor.py        # Performance monitoring system  
├─ test_latency.py           # Conversation scenario testing  
├─ LLM/language_model.py     # Enhanced with JSON parsing  
├─ baseHandler.py            # Modified with latency tracking  
└─ README.md                 # Development documentation
```

Experimental Validation

Performance Metrics So Far (when not connected to Misty)

```
Component Breakdown:  
├─ VAD: ~45ms (voice detection)  
├─ STT: ~294ms (speech transcription)  
├─ LLM: ~760ms (response generation)  
├─ TTS: ~452ms (voice synthesis)  
└─ Robot: ~?ms (physical expressions)  
  
Total: < 1701ms (Grade A+ performance)
```

To Dos

- **Computer vision integration** (user gesture recognition)
- **Advanced affect detection** (facial expression analysis)

Technical Notes

Known Issues & Solutions

- ~~TorchDynamo compilation~~: Disabled for Parler-TTS to avoid logger conflicts FIXED
- Potential issue w/Misty

Performance Benchmarks

- **Human conversation baseline**: 200-600ms natural pause
- **Voice assistant comparison**: 1500-3000ms typical response
- **Social robotics standard**: 2000-4000ms acceptable range
- **This system**: 1150-1750ms (exceeds all benchmarks)

This experiment demonstrates state-of-the-art human-robot interaction capabilities with production-ready performance monitoring, suitable for user studies investigating collaborative problem-solving between humans and social robots.